ПРАКТИЧЕСКАЯ РАБОТА №1

ЭМПИРИЧЕСКИЙ АНАЛИЗ СЛОЖНОСТИ АЛГОРИТМОВ

Часть 1.2: ТЕОРЕТИЧЕСКИЙ МЕТОД ОЦЕНКИ ВЫЧИСЛИТЕЛЬНОЙ СЛОЖНОСТИ. КОРРЕКТНОСТЬ ИТЕРАТИВНОГО АЛГОРИТМА

Цель: актуализация знаний и приобретение практических умений и навыков по определению вычислительной сложности алгоритмов (эмпирический подход) и доказательству корректности итерационных алгоритмов.

Задание 1. Определить эффективный алгоритм из двух предложенных в задании 1 практической работы №1.1, используя теоретический метод оценки сложности каждого из алгоритмов (в рамках эмпирического подхода).
Напомним задачу: дан массив из n элементов, удалить из массива все значения, равные заданному пользователем (ключевому).
Два алгоритма решения этой задачи:

x-массив, n – количество элементов в массиве, key – удаляемое значение

Алгоритм 1.

```
delFirstMetod(x,n,key) {
i←1
while (i<=n) do
        if x[i]=key then
                //удаление
                 for j←i to n-1 do
                  x[j] ←x[j+1]
                od
                n←n-1
       else
              i←i+1
      endif
od
}
```

Алгоритм 2

```
delOtherMetod(x,n,key) {
j←1
for i←1 to n do
  x[j]=x[i];
  if x[i]!=key then
     j++
  endif
od
if x[1]=key then
  n←0
else
  n←j
endif
}
```

Требования к выполнению задания 1:

1. Для каждого алгоритма привести в отчёте следующие результаты:
   1.1. Для каждого из алгоритмов:
      a. На основе анализа псевдокода или кода на C++ определить формулы ***функции роста*** времени работы этих алгоритмов T(n) отдельно, когда а) все элементы в массиве неключевые и б) все элементы в массиве ключевые.
      b. По полученным функциям роста вычислить количество $T_т$ критических операций при n = 10, 100, 1000, 10000.

c. Оценить ***порядок роста*** (скорость роста) функции роста T(n) – ***степень полинома***.

1.2. Представить результаты исследования в таблицах для двух алгоритмов (по примеру табл. 1), указав для каждого n: $T_т$ – количество критических операций согласно теоретическим расчетам и $T_п$ – практически полученное количество операций при выполнении алгоритма (по счётчикам, по данным предыдущей практической работы №1.1).

Таблица 1. Сводная таблица результатов

	Все элементы ключевые		**Все элементы неключевые**	
n	**$T_т$**	**$T_п$**	**$T_т$**	**$T_п$**
10				
100				
1000				
10000				

Примечание: в указанных случаях $T_т$ должно быть равно $T_п$.

2. Сравните результаты подсчета команд, вычисленные здесь по формулам, с результатами по счетчикам в практической работе №1.1. убедитесь в том, что они равны.
   При расхождении результатов определите, когда была допущена ошибка – при практическом подсчёте по счётчикам команд или при теоретическом расчёте по функции роста. Исправьте ошибки.
3. Определите ёмкостную сложность обоих алгоритмов как функции объёма дополнительной памяти от объёма входа.

Задание 2. Оценить теоретически вычислительную сложность алгоритма простой сортировки массива.

1. Вывести на основе псевдокода или кода C++ для любого из двух алгоритмов простой сортировки массива (согласно варианту индивидуального задания практической работы №1.1) формулу функции роста времени работы алгоритма от объёма входа для лучшего и худшего случаев, а также оценить скорость роста функций роста.

2. На основе функций роста рассчитать количество критических операций для объема входа n = 100, 1000, 10000, 100000 и 1000000 элементов. Полученные результаты свести в сводную таблицу по образцу табл. 1 (для ситуаций а) уже упорядоченного массива и б) обратной упорядоченности).

3. Сравните результаты подсчета команд, вычисленные здесь по формулам, с результатами по счетчикам в практической работе №1.1. убедитесь в том, что они равны. Исправьте ошибки, если они есть.

4. Определите ёмкостную сложность алгоритма сортировки как функцию объёма дополнительной памяти от объёма входа.

Задание 3. Докажите корректность алгоритма 2 из задания 1 практической работы №1.1 с помощью ***инварианта цикла***:

1) Сформулируйте инвариант цикла, отражающий желаемый результат.
2) Докажите истинность инварианта на этапах инициализации и сохранения. Что по методу математической индукции приведёт к выводу об истинности инварианта после последней итерации (этап завершения).
3) Докажите конечность цикла с помощью понятия ***области неопределённости***.

По результатам выполнения всех заданий сформируйте отчёт, сохраните его в формате pdf и загрузите в систему СДО.

ЛИТЕРАТУРА:

1. Бхаргава А. Грокаем алгоритмы, 2-е изд. – СПб: Питер, 2024. – 352 с.
2. Ла Рокка М. Грокаем структуры данных. – СПб: Питер, 2026. – 288 с.
3. Кнут Д.Э. Искусство программирования, том 3. Сортировка и поиск, 2-е изд. – М.: ООО «И.Д. Вильямс», 2018. – 832 с.
4. Седжвик Р. Фундаментальные алгоритмы на C++. Анализ/Структуры данных/Сортировка/Поиск. – К.: Издательство «Диасофт», 2001. – 688 с.
5. Алгоритмы – всё об алгоритмах / Хабр [Электронный ресурс]. URL: https://habr.com/ru/hub/algorithms/ (дата обращения 05.02.2025).

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. В чём заключается эмпирический подход к анализу вычислительной сложности алгоритма?
2. Можно ли по скорости роста функции T(n) сравнить эффективность алгоритмов?
3. В чём заключается метод математической индукции?
4. Что такое корректность алгоритма?
5. Что такое инвариант цикла? Может ли условие цикла быть его инвариантом?
6. Как доказать корректность циклического (итеративного) алгоритма с помощью инварианта цикла?
7. Что такое область неопределённости (например, в массиве)? Как с помощью этого понятия доказать конечность цикла?